

Involves the use of Docker, Kubernetes, AWS Cloud and GitHub actions

Flagged a few places as **(Choke point)** as I think it can lead to integration issues.

Overview of Steps

1. **Set up AWS CLI and Kubernetes tools on your local system**
 2. **Create AWS services: EKS Cluster (Kubernetes) and RDS (MySQL)**
 3. **Prepare project structure & create necessary files**
 4. **Containerize backend & frontend using Docker**
 5. **Deploy backend & frontend on Kubernetes**
 6. **Set up CI/CD automation with GitHub Actions**
 7. **Test everything & monitor logs**
-

Step 1: Setup AWS CLI, Kubernetes, and Docker

1.1 Install Required Tools on Your Local System

You will be running commands on **your local system's terminal** (Command Prompt, PowerShell, or Terminal for Mac/Linux).

- ✓ Install **AWS CLI** → [Install Guide](#)
 - ✓ Install **kubectl** (Kubernetes CLI) → [Install Guide](#)
 - ✓ Install **eksctl** (For managing AWS EKS) → [Install Guide](#)
 - ✓ Install **Docker Desktop** → [Install Guide](#)
-

Step 2: Configure AWS on Your Local System

2.1 Log in to AWS from Your Terminal

Run the following command:

`aws configure`

It will ask you for:

- **AWS Access Key ID** → Found in AWS Console > IAM > Security Credentials
- **AWS Secret Access Key** → Found in AWS Console > IAM > Security Credentials
- **Default region** → Use `us-east-1`
- **Output format** → Leave blank (or type `json`)

2.2 Verify AWS Setup

Run:

`aws s3 ls`

If you see a list of buckets (or an empty list), AWS CLI is working.



Step 3: Create AWS Kubernetes Cluster (EKS)

3.1 Create Kubernetes Cluster

Run this command in your **local terminal**:

sh

CopyEdit

```
eksctl create cluster --name my-cluster --region us-east-1  
--nodegroup-name standard-workers --node-type t3.medium --nodes 2
```

This will take **10-15 minutes**. It creates: ☒ **EKS Cluster**

☒ **2 Worker Nodes**

3.2 Verify Kubernetes Cluster

sh

CopyEdit

```
kubectl get nodes
```

If it shows worker nodes (**Ready** status), your cluster is set up.

Step 4: Create AWS MySQL Database (RDS)

4.1 Create MySQL Database

1. **Go to AWS Console → RDS → Create Database**
2. Choose (easy create template):
 - **Engine:** MySQL
 - **Version:** 8.x
 - **Instance Class:** `db.t3.micro` (Free Tier)
 - **Storage:** 20GB
 - **Username:** `admin`
 - **Password:** `yourpassword`
3. **Enable Public Access** (not possible through easy create template, will do it in some time)
4. **Click Create Database**
5. After creation, **copy the Endpoint URL**

Fix: Enable Public Access

1 Modify RDS to Allow Public Access

1. **Go to AWS Console → RDS → Databases**
2. **Click on test-db-codelingo**
3. **Scroll down to Connectivity & security**
4. **Under Public access, click Modify**
5. **Change Publicly accessible to Yes**
6. **Click Continue → Apply changes**

2 Modify Security Group to Allow External Access

Even after enabling public access, your database is still locked by the default security group. You need to allow external access.

Add a Security Group Rule

1. Go to AWS Console → EC2 → Security Groups
2. Find the Security Group linked to test-db-codelingo
3. Click Inbound Rules → Edit Inbound Rules
4. Add a new rule:
 - Type: MySQL/Aurora
 - Protocol: TCP
 - Port: 3306
 - Source: **0.0.0.0/0** (Allow from anywhere) *(For testing only, use a secure IP later)*
5. Click Save Rules

③ Copy Endpoint URL

1. Go to AWS Console → RDS → Databases
2. Click test-db-codelingo
3. Under Connectivity & security, find Endpoint
4. Copy this RDS endpoint URL (e.g.,
test-db-codelingo.csmhdyfqwe.us-east-1.rds.amazonaws.com)

Now that your database is **publicly accessible**, update your Kubernetes secrets.

Note:

The EC2 connection setup option is not required for your deployment because you are connecting your RDS MySQL database to your Kubernetes cluster (EKS), not a standalone EC2 instance.

What This Means:

- "Don't connect to an EC2 compute resource" because your database will be accessed by your EKS cluster and not an individual EC2 instance.
- "Connect to an EC2 compute resource" only needed if your database is meant to be used by an EC2 instance directly.

Step 1: Set Up AWS ElastiCache for Redis

1. Go to AWS Console → ElastiCache → Redis OSS cache

1 Select the Right Deployment Option

You have two options for deployment:

-  **Serverless** → This is a new feature that automatically scales but is more expensive and not needed for your use case.
-  **Design Your Own Cache** → Choose this option to control the instance type and settings.

👉 Click "Design your own cache" instead of "Serverless."

2 Creation Method

-  Choose: "Easy Create" (For recommended best practice settings)
 -  Do NOT choose "Cluster Cache" unless you need horizontal scaling (advanced use case).
-

3 Configuration - Node Type

- For testing/MVP, choose:
 -  Demo - **cache.t4g.micro** (Best for free-tier/small usage)
 - For small production, choose:
 -  Dev/Test - **cache.t4g.large**
-  Avoid "Production" (**cache.r6g.large**) unless needed for high performance (expensive).
-

4 Cluster Info

- Set a Name → Example: **my-redis-cluster**
- Description → Optional, but you can write:
"Redis instance for my application"

5 Connectivity (Important!)

5.1 Network Type

- ✓ Choose IPv4 (default)
- ✗ Do NOT select dual-stack unless you require IPv6 compatibility.

5.2 Subnet Groups

Since Redis needs to be inside the same AWS VPC as your EKS cluster, you need to:

- ✓ Create a New Subnet Group

- Name → Example: **redis-subnet-group**
- Description → "Subnet group for Redis instance"
- Subnet Selection → Choose subnets in the same VPC as your AWS EKS cluster

💡 Why?

- This ensures your backend in Kubernetes can connect to Redis.
- If you pick the wrong subnet, your application will not be able to reach Redis.

6 Create the Redis Cluster

Once you've configured everything, click "Create Cluster" and wait for it to be ready.

7 Get Redis Connection Details

Once the cluster is created:

1. Go to AWS ElastiCache Dashboard
2. Find your Redis instance
3. Since Primary Endpoint is missing, you should use the Configuration Endpoint instead.

Then, Redis "AUTH Default User Access: No" & "Transit Encryption Mode: Required"

- ✓ "AUTH Default User Access: No" → No password is required for authentication.
- ✓ "Transit Encryption Mode: Required" → TLS (SSL) encryption is required for all connections.

What this means:

- Your Redis does NOT require a password for authentication.
 - Your Redis REQUIRES TLS encryption to connect securely.
-



How to Connect to Redis with TLS (SSL)

Since TLS (SSL) is required, your backend must connect to Redis using an encrypted TLS connection.

1 Modify Your Backend Code to Use TLS

Check if your Redis client in Node.js (or any other language) supports TLS connections.

For Node.js, if you're using **ioredis**, update your Redis connection to enable TLS:
(Choke point)

```
const Redis = require("ioredis");

const redis = new Redis({
  host: process.env.REDIS_HOST,
  port: 6379, // Default Redis port
  tls: {} // Enables TLS for secure connection
});

redis.on("connect", () => {
  console.log("Connected to AWS Redis with TLS");
});

redis.on("error", (err) => {
```

```
    console.error("Redis connection error:", err);  
  });
```

```
module.exports = redis;
```

♦ Why is `tls: {}` needed?

- AWS requires encrypted connections (TLS), otherwise the connection will fail.

Why?

- Without TLS, AWS Redis will reject connections.
- This allows your backend to connect securely.

Need to Check If AWS EKS Has Internet Access Before Choosing a Container Registry to push the Docker images for backend and frontend

✅ If EKS has internet access, you can use Docker Hub to store and pull images.

❌ If EKS does NOT have internet access, you must use AWS ECR because EKS cannot pull images from Docker Hub.



Step 1: Check If Your EKS Cluster Has Internet Access

Since your previous test failed due to no backend pods running, let's first check if EKS itself can reach the internet.

1 Create a Test Pod That Uses Curl

Run:

sh

CopyEdit

```
kubectl run internet-test --image=busybox --restart=Never --command --  
sleep 3600
```

✓ This starts a temporary pod that does nothing for 1 hour.

Now connect to this pod and test internet access:

sh

CopyEdit

```
kubectl exec -it internet-test -- wget --spider https://www.google.com
```

✓ If this succeeds, your EKS cluster has internet access.

✗ If this fails, your EKS cluster does NOT have internet access and must use AWS ECR instead of Docker Hub.

As our EKS had internet access I chose the docker hub route because pushing to Docker Hub is simpler and avoids extra AWS setup, as it has tighter security protocols, still a comparison below

Comparison	Docker Hub  (Preferred for You)	AWS ECR (More Secure, But Extra Work)
Ease of Use	 Simple (No AWS IAM setup needed)	 Complex (Needs AWS authentication)
Pulling Images	 Works with your EKS cluster (since it has internet)	 Works even if EKS is private
Security	 Less secure (public images)	 More secure (private AWS access)
Speed	 Fast (if EKS has internet)	 Faster (if EKS is private, inside AWS)
Setup Effort	 Minimal (just push to Docker Hub)	 Requires AWS IAM roles & permissions
Pricing	 Free (unless private repo)	 AWS charges for ECR storage & data transfer

Docker Image to Docker Hub:

Next Steps: Push Your Docker Images to Docker Hub

Now that we know your EKS cluster has internet access, follow these steps:

1 Log in to Docker Hub

```
docker login
```

Enter your Docker Hub username & password when prompted.

2 Build Backend Image

Run this from inside the `docker/` directory:

sh

CopyEdit

```
docker build -t backend -f server.Dockerfile ..
```

✓ Why ..?

- `server.Dockerfile` is in `docker/`, but your project files (like `server/` folder) are in the parent directory.
 - The `..` tells Docker to **include files from the project root**.
-

2 Build Frontend Image

Run this from inside the `docker/` directory:

sh

CopyEdit

```
docker build -t frontend -f client.Dockerfile ..
```

✓ Now, your Docker images are built successfully!



Next Steps: Push Images to Docker Hub

Once the images are built, **tag them for Docker Hub**:

1 Tag Backend Image

sh

CopyEdit

```
docker tag backend your-dockerhub-username/backend:latest
```

2 Tag Frontend Image

sh

CopyEdit

```
docker tag frontend your-dockerhub-username/frontend:latest
```

3 Push Both Images to Docker Hub

First create the `your-dockerhub-username/backend` and `your-dockerhub-username/frontend` repositories in docker hub

```
docker push your-dockerhub-username/backend:latest  
docker push your-dockerhub-username/frontend:latest
```

✅ Now your images are publicly available for Kubernetes to pull.

Doing the above would later lead to issues in kubernetes not being able to pull docker image

Because of Architecture mismatch between your image and AWS EKS nodes.

Most AWS EKS nodes run on `amd64`, while your local machine (MacBook) has built the image as `arm64`.



Step 1: Check Image Architecture

Run:

sh

CopyEdit

```
docker inspect your-dockerhub-username/backend:latest | grep  
Architecture
```

✅ Expected output:

json

CopyEdit

```
"Architecture": "amd64"
```



Step 2: Build & Push a Multi-Platform Image

If your image is not `amd64`, you need to rebuild and push it for multiple architectures.

1 Enable Multi-Platform Builds in Docker

sh

CopyEdit

```
docker buildx create --use
```

2 Build and Push for Both `amd64` and `arm64`

sh

CopyEdit

```
docker buildx build --platform linux/amd64,linux/arm64 -t  
your-dockerhub-username/backend:latest --push -f  
docker/server.Dockerfile .
```

3 Do the Same for Frontend

sh

CopyEdit

```
docker buildx build --platform linux/amd64,linux/arm64 -t  
your-dockerhub-username/frontend:latest --push -f  
docker/client.Dockerfile .
```

✓ This will ensure Kubernetes (which runs on `amd64`) can pull the image correctly.



Step 3: Restart Kubernetes Deployment

Once images are correctly built & pushed, force Kubernetes to pull them again.

1 Delete Failing Pods

sh

CopyEdit

```
kubectl delete pod --selector=app=backend -n code-learning
```

```
kubectl delete pod --selector=app=frontend -n code-learning
```

2 Apply Kubernetes Deployments Again

sh

CopyEdit

```
kubectl apply -f k8s/backend-deployment.yaml
```

```
kubectl apply -f k8s/frontend-deployment.yaml
```

3 Check If Pods Are Running

sh

CopyEdit

```
kubectl get pods -n code-learning
```

Kubernetes setup -

Step 1: Create a **k8s/** Directory

In your project root, create a directory for Kubernetes YAML files:

```
sh
CopyEdit
mkdir k8s
cd k8s
```

In a separate directory named "k8s" -

Step 2: Create Kubernetes Namespace

A namespace helps keep things **organized**.

- ♦ Create **k8s/namespace.yaml**

```
yaml
CopyEdit
apiVersion: v1
kind: Namespace
metadata:
  name: code-learning
```

Apply:

```
sh
CopyEdit
kubectl apply -f k8s/namespace.yaml
```

Step 3: Store AWS MySQL & Redis Credentials in Kubernetes Secrets

Since your **database and Redis are in AWS**, store their credentials **securely**.

Now, run the following **with your updated values**:

Recreate MySQL Secret

```
sh
CopyEdit
kubectl create secret generic mysql-secret \
  --namespace code-learning \
  --from-literal=MYSQL_USER=admin \
  --from-literal=MYSQL_PASSWORD=yourpassword \

--from-literal=MYSQL_HOST=test-db-codelingo.cbmko46okb96.us-west-1.rds
.amazonaws.com \
  --from-literal=MYSQL_DATABASE=code_learning_platform
```

2 Redis Secret (Without Port)

Your previous Redis secret included `:6379` in the `REDIS_HOST` value.
AWS ElastiCache **does not require** specifying the port, so remove `:6379`:

```
sh
CopyEdit
kubectl create secret generic redis-secret \
  --namespace code-learning \

--from-literal=REDIS_HOST=clustercfg.my-redis-cluster.rhesam.usw1.cach
e.amazonaws.com
```

Step 3: Verify That Secrets Are Created Correctly

Run:

```
sh
CopyEdit
kubectl get secrets -n code-learning
```

To **check the values stored** (without revealing passwords):

```
sh
CopyEdit
kubectl describe secret mysql-secret -n code-learning
kubectl describe secret redis-secret -n code-learning
```


Delete Existing Secrets if needed but will have to recreate again

Run the following commands to remove the secrets **from the code-learning namespace**:

```
sh
CopyEdit
kubectl delete secret mysql-secret -n code-learning
kubectl delete secret redis-secret -n code-learning
```

✓ Review of Your `k8s/backend-deployment.yaml` and `k8s/frontend-deployment.yaml`

Your Kubernetes deployment and service configurations **look good overall!** However, I have a few **important improvements and fixes** for better stability and performance.

Recommended Changes to `k8s/backend-deployment.yaml`

1 Missing `imagePullPolicy`

- Add `imagePullPolicy: Always` to ensure Kubernetes always pulls the latest Docker image.
- Without this, Kubernetes may use **old cached images** instead of pulling the latest.

2 Add Liveness & Readiness Probes

- These help Kubernetes restart the pod if the application crashes.
- The **readiness probe** ensures Kubernetes only sends traffic when the backend is fully started.

3 Add Resource Limits

- Prevents backend pods from consuming too much memory/CPU.
-

✓ Fixed k8s/backend-deployment.yaml

yaml

CopyEdit

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: code-learning
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: your-dockerhub-username/backend:latest
          imagePullPolicy: Always # Ensures Kubernetes always pulls the
latest image
          ports:
            - containerPort: 3001
          env:
            - name: DB_HOST
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_HOST
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
```

```
        key: MYSQL_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: MYSQL_PASSWORD
- name: DB_NAME
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: MYSQL_DATABASE
- name: REDIS_HOST
  valueFrom:
    secretKeyRef:
      name: redis-secret
      key: REDIS_HOST
- name: JWT_SECRET
  value: your_jwt_secret_key
- name: NODE_ENV
  value: production
```

```
livenessProbe:
  httpGet:
    path: /health
    port: 3001
  initialDelaySeconds: 10
  periodSeconds: 10
```

```
readinessProbe:
  httpGet:
    path: /health
    port: 3001
  initialDelaySeconds: 5
  periodSeconds: 5
```

```
resources:
  limits:
    memory: "512Mi"
```

```
        cpu: "500m"
      requests:
        memory: "256Mi"
        cpu: "250m"
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: code-learning
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 3001
      targetPort: 3001
  type: ClusterIP
```

✅ Why These Changes?

1. **imagePullPolicy: Always** → Ensures Kubernetes **always pulls the latest image** instead of using old ones.
 2. **livenessProbe & readinessProbe** → Ensures Kubernetes restarts the container if the backend crashes.
 3. **Resource Limits (limits and requests)** → Prevents backend from using too much CPU/memory.
-

Recommended Changes to k8s/frontend-deployment.yaml

1 Missing imagePullPolicy

- Add **imagePullPolicy: Always** to ensure Kubernetes always pulls the latest Docker image.

2 Fix Backend API URL

Your frontend `REACT_APP_API_URL` is currently:

yaml

CopyEdit

`http://backend-service.code-learning:3001`

-

Instead, **use `backend-service` directly (without namespace):**

yaml

CopyEdit

`http://backend-service:3001`

-

- This makes it work **inside the same Kubernetes namespace**.

✓ Fixed `k8s/frontend-deployment.yaml`

yaml

CopyEdit

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: frontend
```

```
  namespace: code-learning
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: frontend
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: frontend
```

```
    spec:
```

```
      containers:
```

```
        - name: frontend
```

```
          image: your-dockerhub-username/frontend:latest
```

```
          imagePullPolicy: Always # Ensures Kubernetes always pulls the
```

```
latest image
```

```
          ports:
```

```

    - containerPort: 3000
  env:
    - name: REACT_APP_API_URL
      value: http://backend-service:3001 # FIXED backend service
URL

  resources:
    limits:
      memory: "512Mi"
      cpu: "500m"
    requests:
      memory: "256Mi"
      cpu: "250m"
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: code-learning
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
  type: LoadBalancer

```

✅ Why These Changes?

1. **imagePullPolicy: Always** → Ensures **latest image is pulled** in Kubernetes.
2. **Fixed REACT_APP_API_URL** → Now correctly points to `http://backend-service:3001`.
3. **Resource Limits** → Prevents frontend from over-consuming CPU/memory.

Next Steps

1 Apply Kubernetes Deployments

sh

CopyEdit

```
kubectl apply -f k8s/backend-deployment.yaml
```

```
kubectl apply -f k8s/frontend-deployment.yaml
```

2 Verify if Pods are Running

sh

CopyEdit

```
kubectl get pods -n code-learning
```